

Projeto de Busca e Ordenação

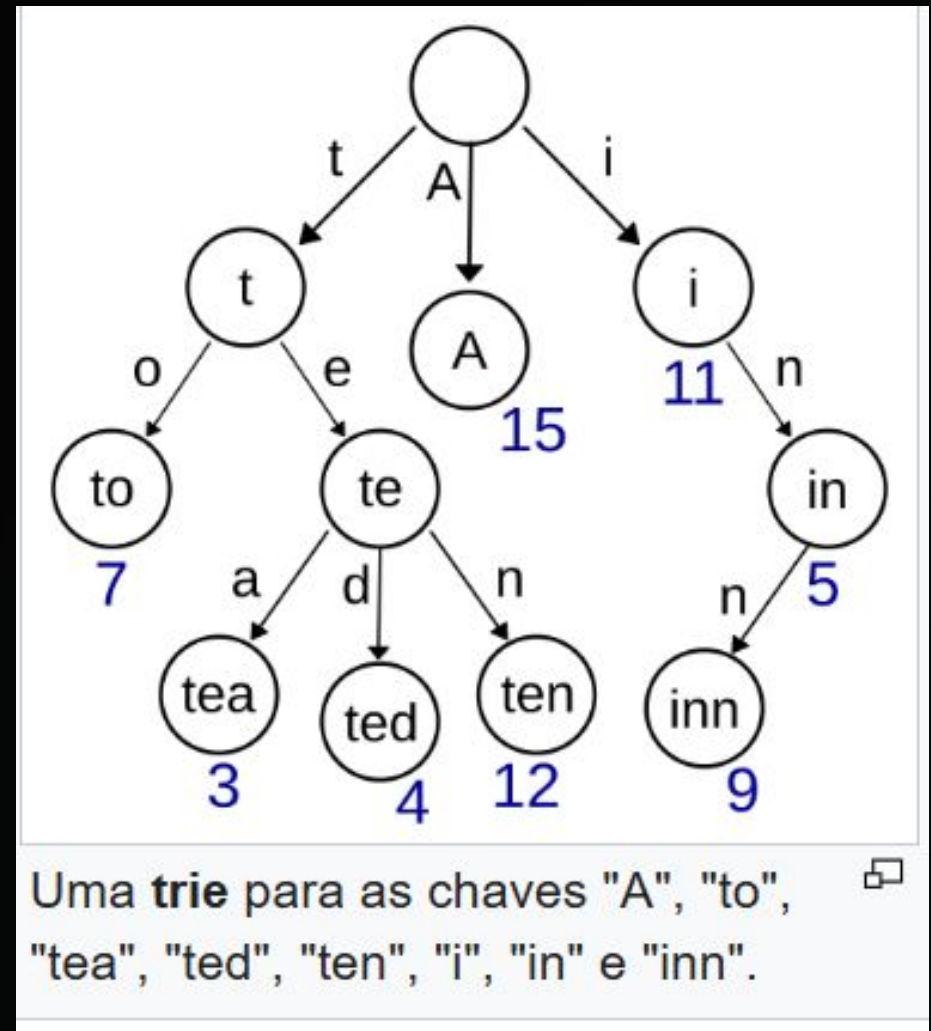
Verificador Ortográfico

ESTRUTURA DE DADOS: TRIE

Por que usar uma Trie?

Uma árvore de prefixos (Trie) é ideal para verificadores ortográficos:

- ⚡ **Busca $O(L)$:** O tempo depende apenas do tamanho da palavra (L), não do dicionário.
- 🔗 **Prefixos Compartilhados:** Economia de espaço ao armazenar palavras com raízes comuns.
- ✓ **Ideal para Sugestões:** Facilita a implementação futura de sugestões por distância de edição.



TRIENODE



Array de Filhos

Cada nó contém um array de 26 posições (a-z), representando as próximas letras possíveis na sequência da palavra.



Flag FimDePalavra

Uma variável booleana que indica se o caminho percorrido até aquele nó específico forma uma palavra completa no dicionário.



Construtor

Inicializa todos os ponteiros como nullptr e a flag como false, garantindo segurança na alocação de memória.

A CLASSE DICCIONARIO

Função Inserir()

Percorre a árvore letra por letra. Se o nó para a próxima letra não existe, ele é criado dinamicamente.

Função Buscar()

Navega pelos nós seguindo os caracteres da palavra. Se encontrar um nullptr, a palavra está errada. Caso contrário, valida via flag.

DESAFIO : O VERIFICADOR ORTOGRÁFICO (TRIE)

```
class Dicionario {
private:
    struct TrieNode {
        TrieNode* filhos[26];
        bool fimDePalavra;
        TrieNode() {
            fimDePalavra = false;
            for(int i=0; i<26; i++) filhos[i] = nullptr;
        }
    };
    TrieNode* raiz;
public:
    Dicionario() { raiz = new TrieNode(); }
    void inserir(string palavra);
    bool buscar(string palavra);
};
```

Modelagem Orientada a Objetos

Para suportar um dicionário de alta volumetria, encapsulamos uma estrutura **Trie** dentro da classe controladora.

-  **Estrutura Interna Aninhada:** O TrieNode é privado, impedindo manipulações externas inseguras de ponteiros.
- A Alfabeto Fixo:** Alocação estática de tamanho 26 para indexação direta $O(1)$ por caractere calculando caractere - 'a'.

DESAFIO : INSERÇÃO NO DICIONÁRIO

```
void Dicionario::inserir(string palavra) {  
    TrieNode* atual = raiz;  
    for (char c : palavra) {  
        int indice = c - 'a';  
        if (atual->filhos[indice] == nullptr) {  
            atual->filhos[indice] = new TrieNode();  
        }  
        atual = atual->filhos[indice];  
    }  
    atual->fimDePalavra = true;  
}
```

População Dinâmica da Árvore

O método consome a palavra caractere por caractere descendo os níveis da árvore.

 **Alocação On-Demand:** Nós novos só são instanciados via palavra inédita (new TrieNode()).

 **Sinalização Estrita:** A flag fimDePalavra = true isola palavras que contêm subpalavras idênticas dentro de si (ex: "sol" dentro de "soldado").

Complexidade: $O(L)$

DESAFIO : CONSULTA ORTOGRÁFICA INSTANTÂNEA

```
bool Dicionario::buscar(string palavra) {
    TrieNode* atual = raiz;
    for (char c : palavra) {
        int indice = c - 'a';
        if (atual->filhos[indice] == nullptr) {
            return false; // Erro ortográfico
        }
        atual = atual->filhos[indice];
    }
    return (atual != nullptr && atual->fimDePalavra);
}
```

Verificação em Tempo Constante Algorítmico

A pesquisa de erros ortográficos não realiza iterações em laço sobre o volume do dicionário.

- 🚫 **Early Return:** Ao menor sinal de um ponteiro nulo (nullptr), o programa mata a busca e aponta erro de digitação imediatamente.
- 🚩 **Dupla Validação:** Checar se o caminho existe E se a flag de terminação está ativa garante precisão contra prefixos incompletos.

DESAFIO CRÍTICO: DESALOCAÇÃO E DESTRUTOR

```
void liberarMemoria(TrieNode* no) {  
    if (no == nullptr) return;  
    for (int i = 0; i < 26; i++) {  
        liberarMemoria(no->filhos[i]);  
    }  
    delete no;  
}  
Dicionario::~Dicionario() {  
    liberarMemoria(raiz);  
}
```

Evitando Memory Leaks

A alocação massiva de nós na heap exige limpeza minuciosa de memória.

 **Percurso Pós-Ordem:** A recursão desce até os nós folha mais profundos. Deleta primeiro os filhos para depois poder deletar o nó pai sem perder a referência.

 **Automatizado via RAI:** Integrado diretamente no destrutor de Dicionario.

LIMPEZA DE DADOS

Normalização Textual

O método limparPalavra é vital para a precisão do verificador:

- ↴ Conversão para minúsculas.
- ✂ Remoção de pontuação e caracteres especiais.
- A-Z Tratamento de acentuação (á → a, ç → c) para compatibilidade ASCII.



| DESAFIO DE ROBUSTEZ: HIGIENIZAÇÃO DE STRINGS

```
string limparPalavra(string palavra) {  
    string resultado = "";  
    for (char c : palavra) {  
        if (isalpha(c)) {  
            resultado += tolower(c);  
        }  
        // Tratamento adicional para remover  
        // acentuações e caracteres estendidos  
    }  
    return resultado;  
}
```

Tratamento de Entrada de Arquivo

Arquivos de texto vêm com sujeiras (pontuação, quebras, maiúsculas) que quebrariam o índice do array da Trie [0-25].

▼ `isalpha()` filtra números, símbolos e pontuações do arquivo.

⇅ `tolower()` remove sensibilidade a caixa, padronizando a busca.

CICLO DE EXECUÇÃO

Carregamento



Dicionário lido de arquivo e
inserido na Trie.

Processamento



Texto alvo é lido palavra por
palavra.

Validação



Consulta à Trie e emissão de
alertas de erro.

Cleanup



Liberação recursiva da
memória alocada.

COMPARATIVO DE COMPLEXIDADE



N = No de palavras no dicionário | L = Comprimento da palavra pesquisada.

Mapeamento Algorítmico

DESAFIO : Knuth-Morris-Pratt (KMP)

```
void computarLPS(string pat, int* lps) {  
    int tam = 0; lps[0] = 0;  
    int i = 1;  
    while (i < pat.length()) {  
        if (pat[i] == pat[tam]) {  
            tam++; lps[i] = tam; i++;  
        } else {  
            if (tam != 0) tam = lps[tam - 1];  
            else { lps[i] = 0; i++; }  
        }  
    }  
}
```

Vetor de Prefixos (LPS)

Construção do vetor de falhas pré-processado para o casamento de strings exato.

 **Lógica do Código:** O ponteiro tam rastreia o comprimento do maior prefixo que também é sufixo.

 **Evita Retrocesso:** Se ocorre um descasamento no texto principal, o algoritmo consulta lps[] para saber para onde saltar no padrão sem reanalisar caracteres.

DESAFIO : O LOOP DE BUSCA KMP

```
void buscarKMP(string txt, string pat) {
    int M = pat.length(), N = txt.length();
    int lps[M]; computarLPS(pat, lps);
    int i = 0, j = 0;
    while (i < N) {
        if (pat[j] == txt[i]) { j++; i++; }
        if (j == M) {
            cout << "Padrão encontrado no índice " << i - j;
            j = lps[j - 1];
        } else if (i < N && pat[j] != txt[i]) {
            if (j != 0) j = lps[j - 1];
            else i++;
        }
    }
}
```

Casamento em Tempo Linear

Garante varredura do texto de entrada sem retroceder o índice i .

↔ i avança continuamente.

🎯 Ao achar uma ocorrência inteira ($j == M$), armazena a posição e reinicia j com base no vetor de falha para pegar sobreposições.

Complexidade: $O(N + M)$

| DESAFIO : Correção Ortográfica (Módulo Corretor)

(Módulo Corretor)

↔ Implementação da Distância de Levenshtein por meio de Programação Dinâmica bidimensional.

🎯 Quando a Trie detecta uma palavra inválida, a matriz calcula os custos mínimos de inserção, deleção e substituição de caracteres.

Inserção:

casa para casas distância = 1)

Remoção:

casas para casa, distância = 1)

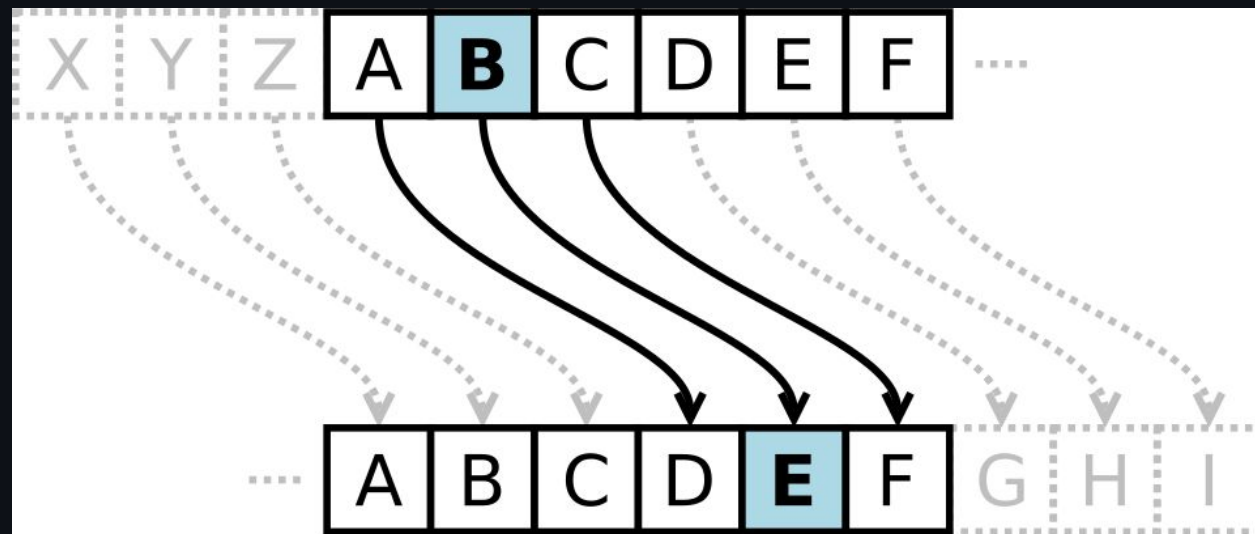
Substituição:

casa para caso, distância = 1). [1, 2]

| DESAFIO : Criptografia e Quebra de Cifra Sem Chave

(Módulo Cifra)

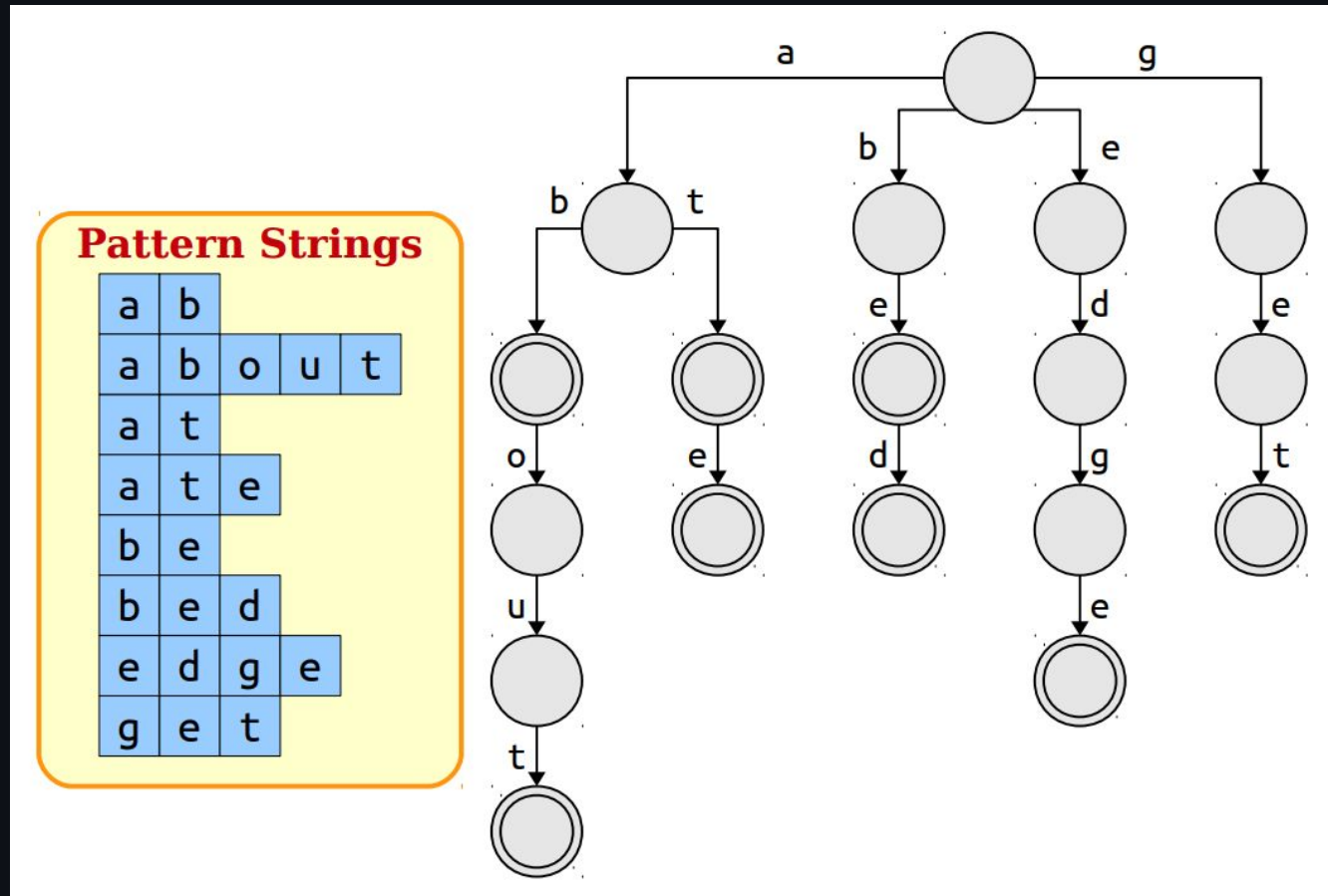
- ↔ A quebra automática avalia todas as 25 rotações possíveis do alfabeto
- 🎯 O texto resultante é quebrado em tokens e validado via Busca Binária ($O(\log N)$) contra o dicionário. Para permitir a Busca Binária, o dicionário é ordenado na inicialização por meio de uma rotina nativa de MergeSort implementada do zero.



DESAFIO : Busca Paralela de Múltiplos Padrões

(Módulo Buscador)

- ↔ Uso do algoritmo de Aho-Corasick. O sistema constrói um autômato de estados ligando os nós da Trie a links de falha (fail links)
- 🕒 busca em largura (BFS iterativa com fila alocada manualmente). Isso permite que centenas de palavras-chave sejam rastreadas em paralelo



DESAFIOS



Marcação

Implementação base de verificação léxica. Identificação imediata de termos ausentes no dicionário Trie.



Sugestão

Algoritmos de aproximação utilizando Distância de Levenshtein para sugerir correções ao usuário.



KMP Core

Busca exata de padrões no texto com pré-processamento de vetor de falhas (LPS) em tempo linear.

FUNCIONALIDADES **ATIVAS**

- ✓ **Dicionário Dinâmico:** Carregamento de arquivos TXT massivos sem perda de performance.
 - ✓ **Filtro de Ruído:** Higienização de strings ignorando pontuação e normalizando caixa alta/baixa.
 - ✓ **Análise de Frequência:** Suporte para contagem de ocorrências via casamento de strings.
 - ✓ **Interface de Terminal:** Saída formatada com alertas visuais de erros ortográficos detectados.
-

DECISÕES DE PROJETO



Memória Manual

Foco total em gestão via new e delete. Controle absoluto sobre o ciclo de vida dos nós da árvore.



Sem utilização de bibliotecas auxiliares

Restrição de bibliotecas prontas para algoritmos. Lógica de busca e ordenação escrita personalizada

EVOLUÇÃO: AHO-CORASICK

O Desafio Máximo

Transição da busca simples para a busca multi-padrão paralela.

- ✓ Construção de Máquina de Estados Finita.
- ✓ Links de Falha entre nós de diferentes ramificações.
- ✓ Performance superior para varredura de múltiplos termos em um único passo.

Obrigado pela atenção!

Demonstração
